论文解读 · arXiv 2611.06918 · 2026-11-04

From Pixels to Actions

Towards Native Vision-Language-Action Policies

NEO-VLA：不用视觉编码器，让动作直接读取骨干网络各层的视觉计算

Weichen Fan, Haiwen Diao, Penghao Wu, …, Dahua Lin, Ziwei Liu
S-Lab, Nanyang Technological University · SenseTime Research · Ropedia

无编码器 2B 骨干　逐层 KV 读取　Flow-matching 专家

一句话：让动作直接读取原生骨干的逐层视觉计算

NEO-VLA 去掉单独预训练的视觉编码器，像素经轻量卷积 patch 化后与语言一起在 2B 、 40 层骨干中演化； flow-matching 动作专家可以逐层读取骨干的 KV 。

发现：对控制最关键的视觉信号往往在早期层就达到峰值，而不在最终表征里；原生骨干冻结时就比编码器式骨干更适合学动作。

90.9

冻结骨干 · LIBERO 平均

同专家、同数据下： Qwen3-VL-2B 79.5 ， Gemma-3-4B 84.5

94.9

冻结时只读前 28 层

高于均匀读 28 层的 90.9 和只读后 28 层的 86.7

89.7

LIBERO-plus 总分

表中最高；相机扰动 94.5 （π0.5 为 78.4 ）

69.4

RoboTwin 2.0 C2R

随机化场景表中最高； Clean 75.78 ，随机化后只掉约 6 分

动机：主流 VLA 只把视觉编码器的最终输出交给动作

主流做法

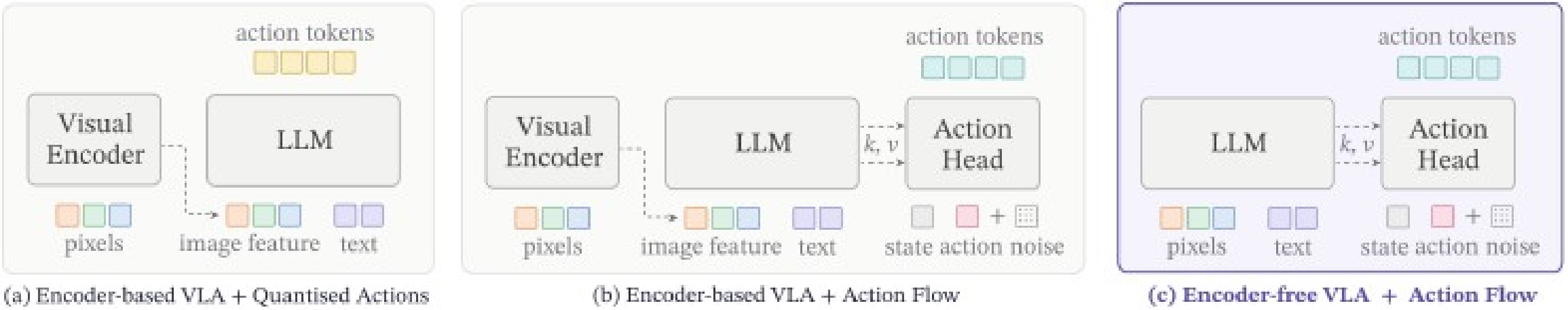
图像 → CLIP / SigLIP / DINOv2 → 语言骨干 → 动作头。
动作只能看到编码器的最终输出，产生它的视觉层级被隐藏。

隐患

操作依赖局部几何、边界、深度、细微状态变化，这些信号最容易被抽象掉。
已有证据：视觉模块是瓶颈、对剪枝敏感、动作微调会损伤视觉表征。

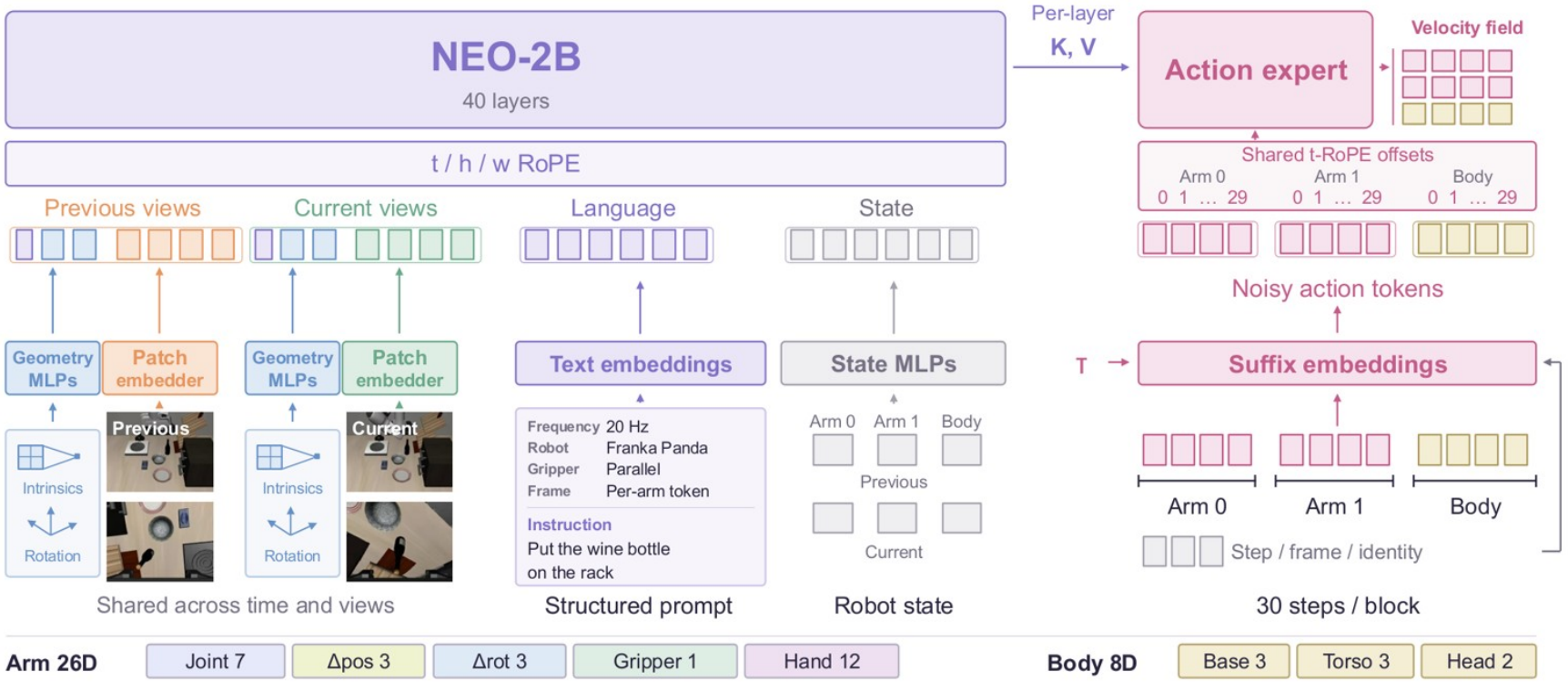
核心问题

动作策略应该继承一个固定的视觉终点，还是直接访问不断演化的视觉计算？



(a) 编码器 + 离散动作 token　　(b) 编码器 + 连续动作头　　(c) NEO-VLA：无编码器，动作专家读取逐层 KV，flow matching 输出连续动作

NEO-VLA：像素进，连续动作出，中间没有视觉编码器



原生视觉接口

两阶段卷积 patch embedder，总步长 32；保持宽高比，每视角 ≤ 88 token；无 ViT / VAE / VQ 码本

共享骨干

NEO-2B， 40 层； t/h/w Native RoPE；文本因果注意力 + 视觉块内双向注意力

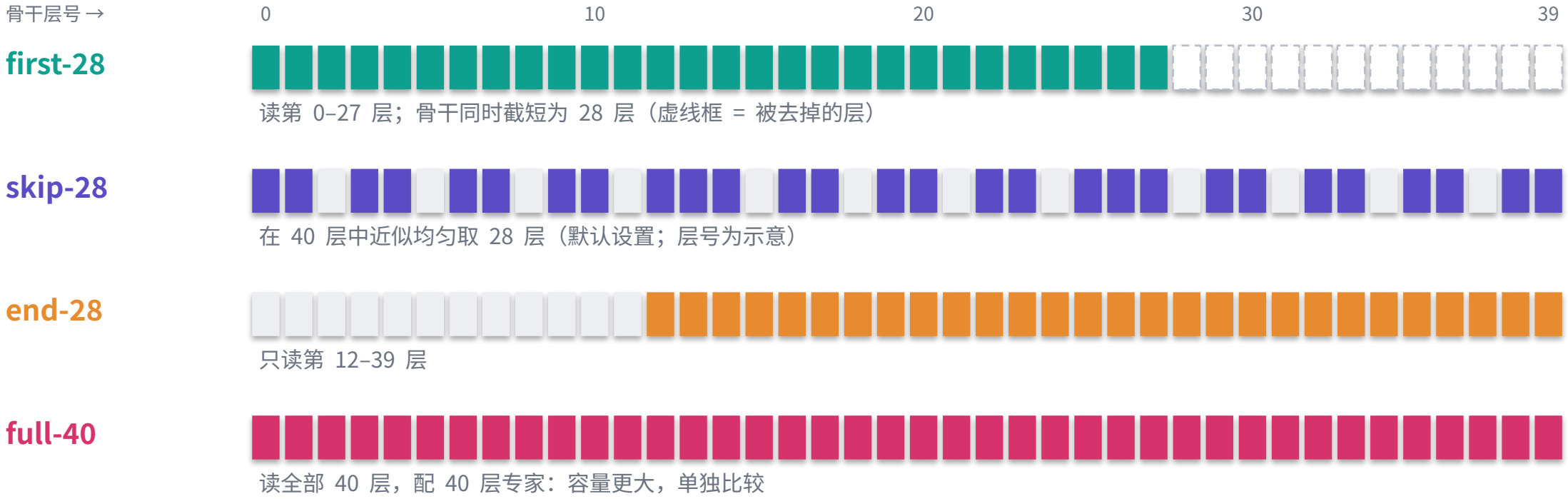
统一多模态前缀

多视角 × 两帧 + 相机内参 / 旋转 MLP；结构化 prompt （频率 / 本体 / 坐标系 / 指令）； 6 个状态 token

Flow-matching 专家

逐层读取骨干 KV；手臂 2×30×26 + 身体 30×8；推理 10 步 Euler，不用 CFG

关键设计：动作专家读哪几层，是一个可配置的选项



Flow matching 训练目标

$a_\tau = (1-\tau)\cdot\varepsilon + \tau\cdot a$，目标速度 $v = a - \varepsilon$

$\tau = 0.999u$， $u \sim \mathrm{Beta}(1, 1.5)$；四组损失加权 (2, 1, 1, 1)，末端执行器与夹爪组权重为 2

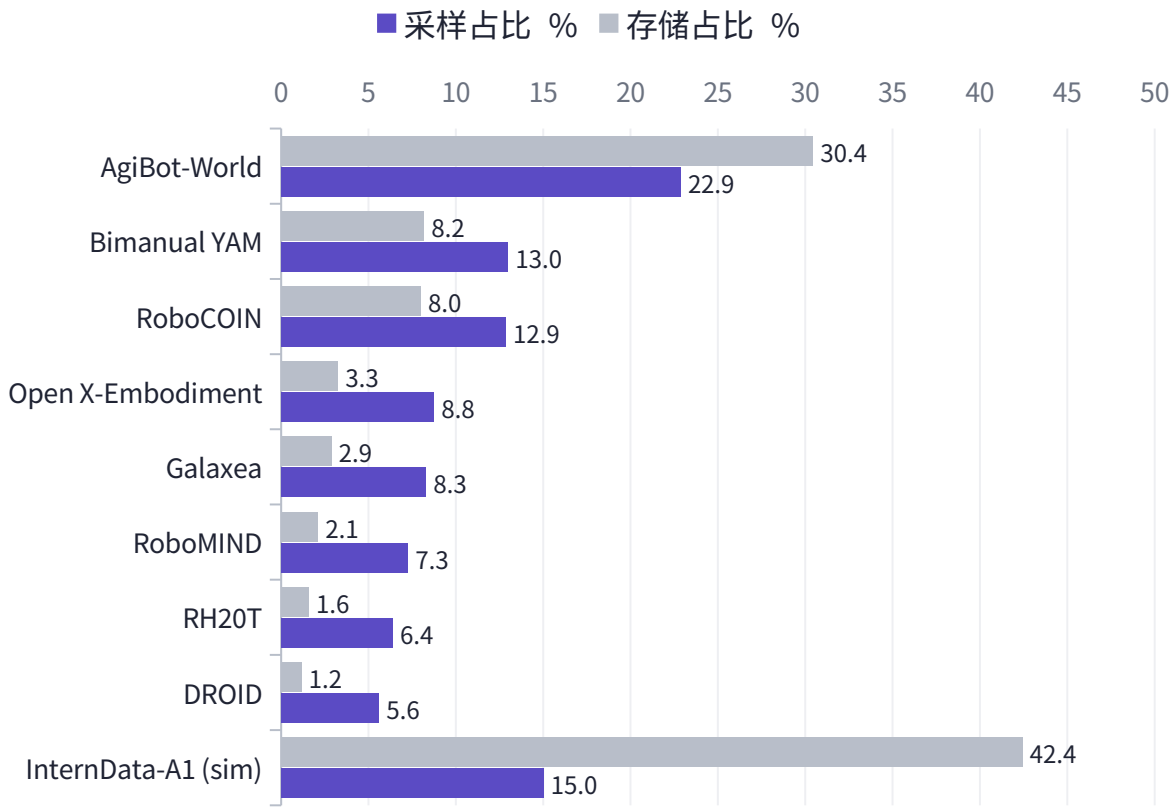
统一动作空间（30 步动作块）

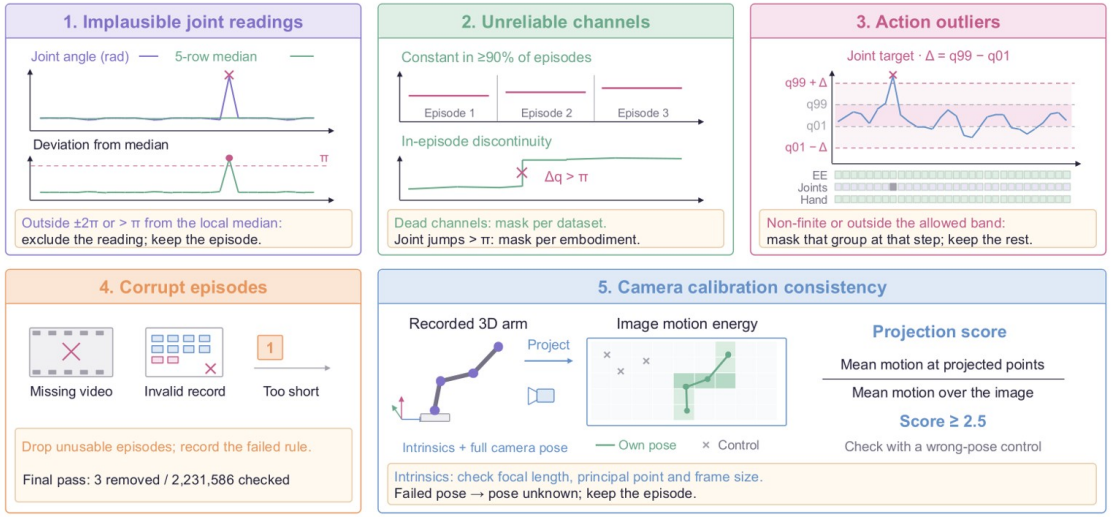
每臂 26 维： 7 关节 +3 平移增量 +3 旋转增量 +1 夹爪 +12 手部关节

身体 8 维： 3 底座 +3 躯干 +2 头部；缺失的部件 mask 掉，适配不同本体

数据： 9 个公开数据集， 223 万条轨迹，约 9,928 小时

2.23M	9,928 h	928M
episodes	时长	行（rows）





清洗原则：只 mask 出问题的部分，不整条丢弃

- 关节读数超出 ±2π 或偏离 5 行局部中位数 >π： 剔除该读数
- ≥90% episode 恒定的通道按数据集 mask； 跳变 >π 按本体 mask
- 动作越界或非有限值：只 mask 该组、该时间步
- 相机位姿用投影运动打分（≥ 2.5 可信，并用错误位姿做对照），失败标记为 unknown
- **最终一轮检查： 2,231,586 条中只删除 3 条**

训练：先冻结骨干训专家，再全参联合训练

① 视觉 - 语言骨干（NEO-2B， 40 层，无编码器）

大规模图文预训练
建立视觉感知，保留语言能力

Mid-training
caption / 对话 / OCR / 检测

指令微调 SFT
高质量多模态交互

② 动作训练（机器人语料）

Stage 1 · 冻结骨干

- 只训动作专家与条件模块
- 113,400 步 · batch 8192
- 峰值 lr 2e-4

Stage 2 · 全参 + 联合训练

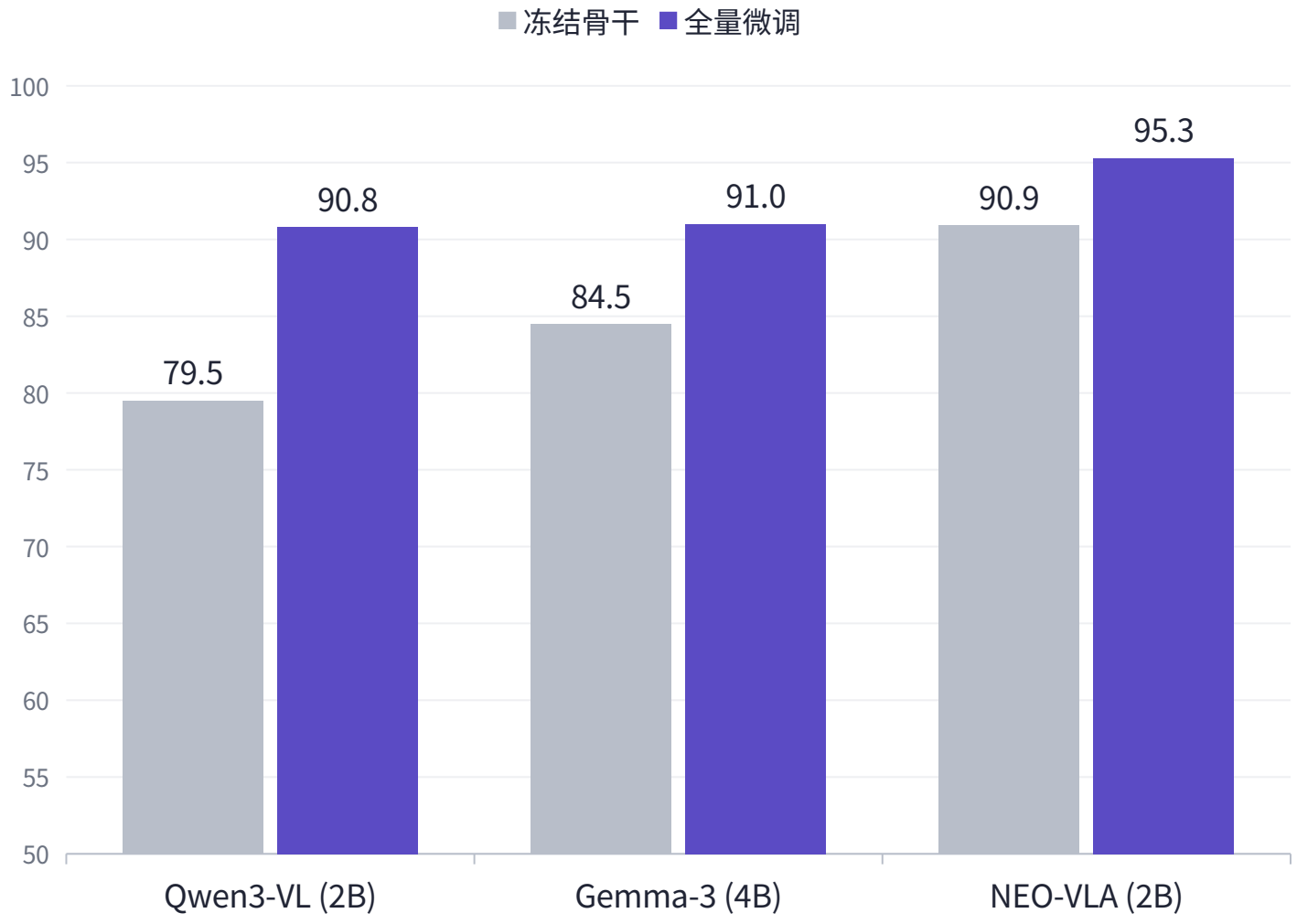
- 解冻全部参数，同一机器人语料
- 75,000 步 · batch 8192
- 专家 lr 2e-4，骨干 lr 5e-5
- 隔步混入理解数据， NTP 损失 ×0.1

Stage 3 · 内部数据

- 从 Stage 2 初始化，全参微调
- 改用内部动作数据
- 75,000 步，其余设置同 Stage 2

优化器 AdamW （β1=0.9, β2=0.95）， 2k 步 warmup 后 cosine 衰减至峰值的 10%， bf16 混合精度，梯度裁剪 1.0

同等条件下，冻结的原生骨干已接近编码器模型全量微调



冻结 NEO ≈ 编码器模型全量微调

90.9 vs 全量微调 90.8 / 91.0

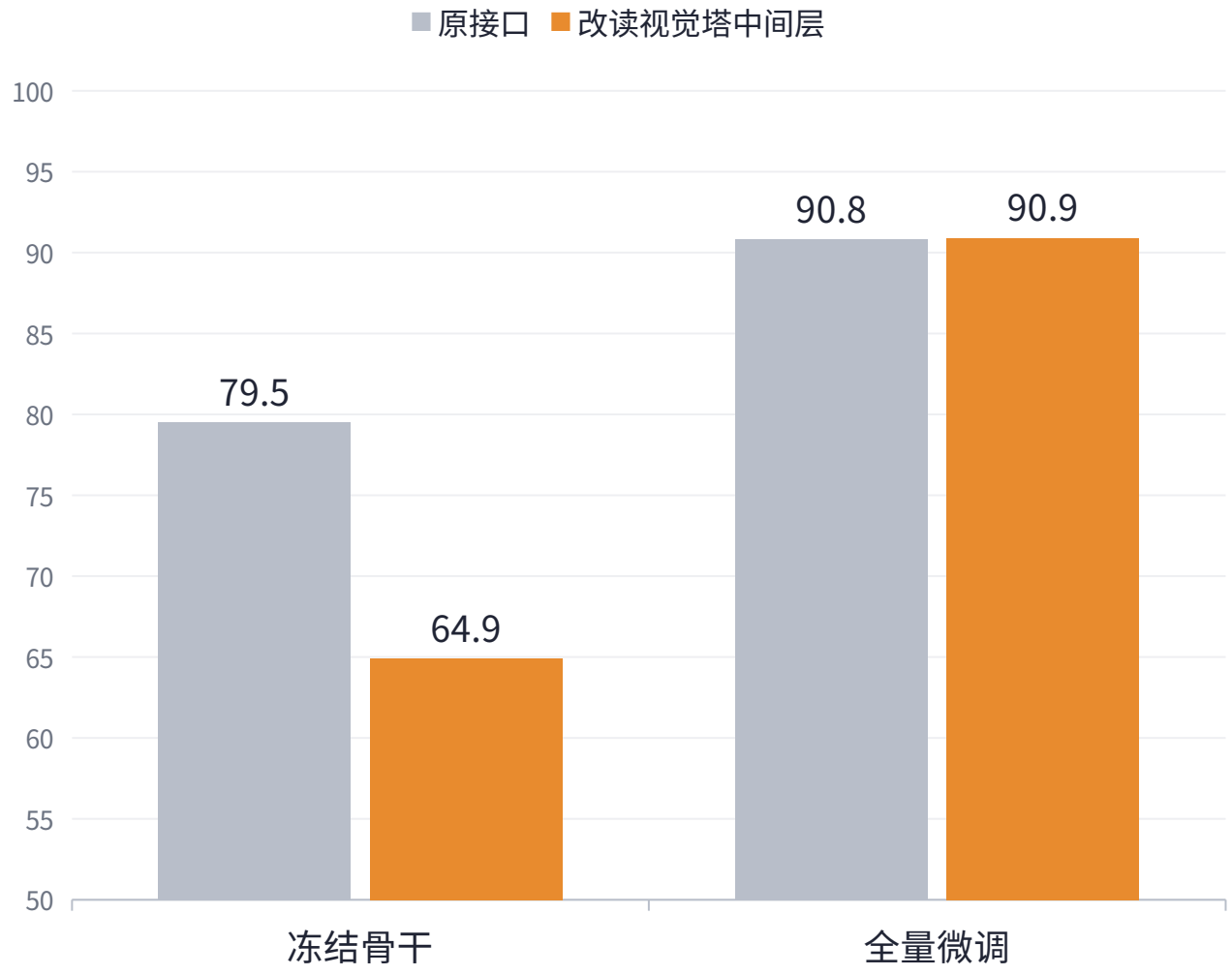
长时序任务差距最大

LIBERO-Long 冻结 85.4 > Qwen / Gemma 全量微调 78.6 / 82.8

注意混杂因素

三个骨干的预训练数据和方法不同，作者承认不能把收益全部归因于去掉编码器。统一设置： OXE 动作预训练 75k 步，评测取 30k 步 checkpoint，每个配置 2,000 回合。

反例：让 Qwen 直接读视觉塔中间层，冻结时反而更差



做法

Qwen3-VL-2B 的 13 个专家层改为读取视觉塔的中间特征，不增加任何参数，其余设置相同。

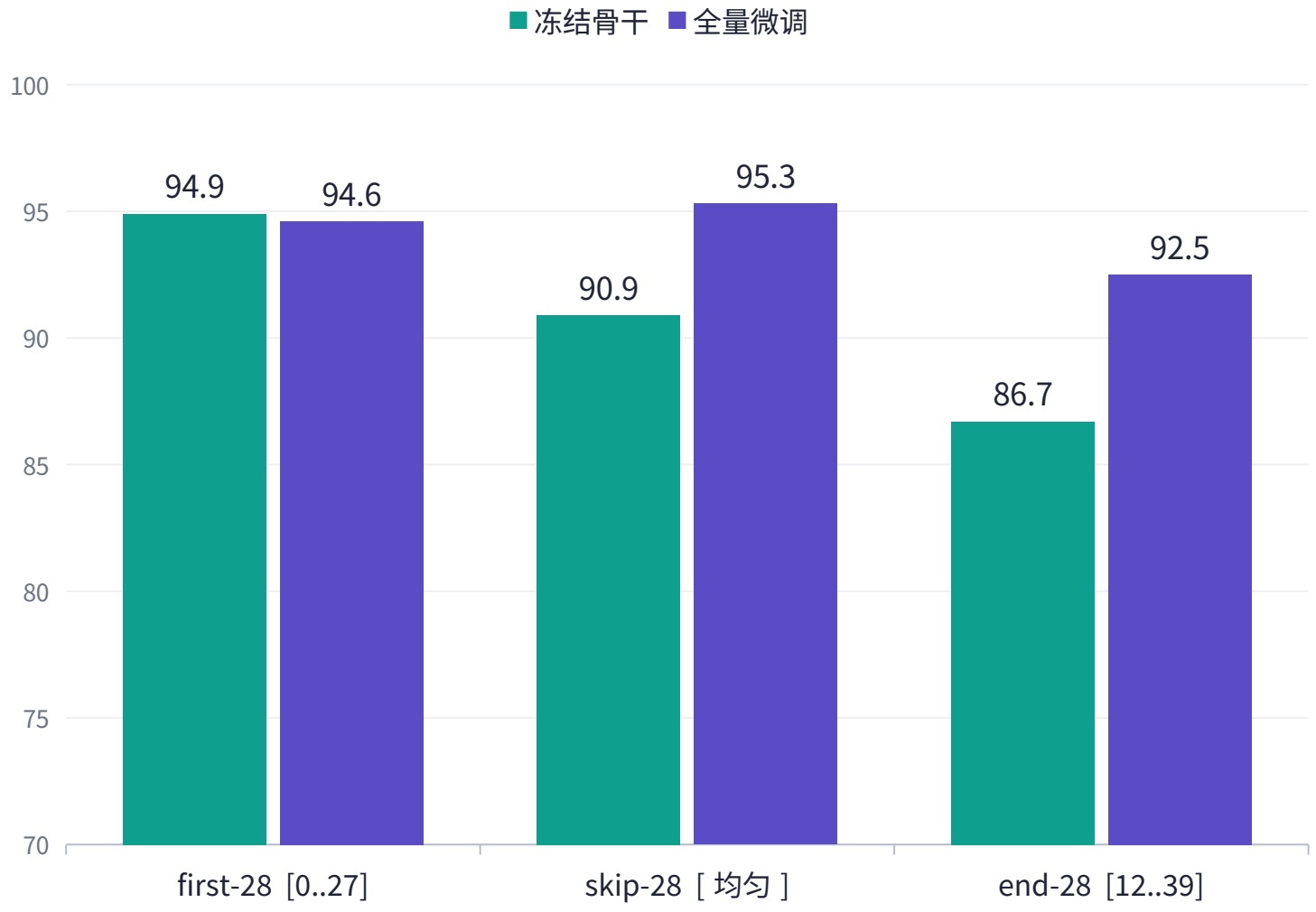
−14.6

冻结时成绩下降（79.5 → 64.9）；全量微调后持平（90.8 vs 90.9）

论文结论

NEO 的优势不只在于“能读到早期特征”，而在于编码器无关的预训练把这些特征塑造成了可直接用于控制的表征。

读取深度消融：冻结时早期层最有用，微调后早晚层结合最好



不重叠对照（冻结）

93.6
first-14 [0..13]

89.6
last-14 [26..39]

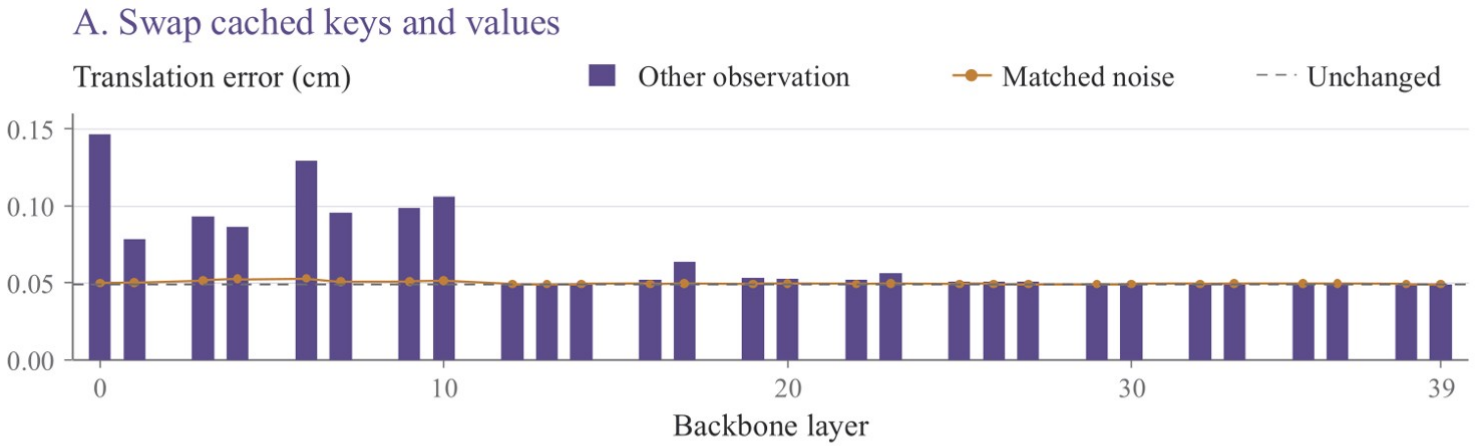
两种设置下结论不同

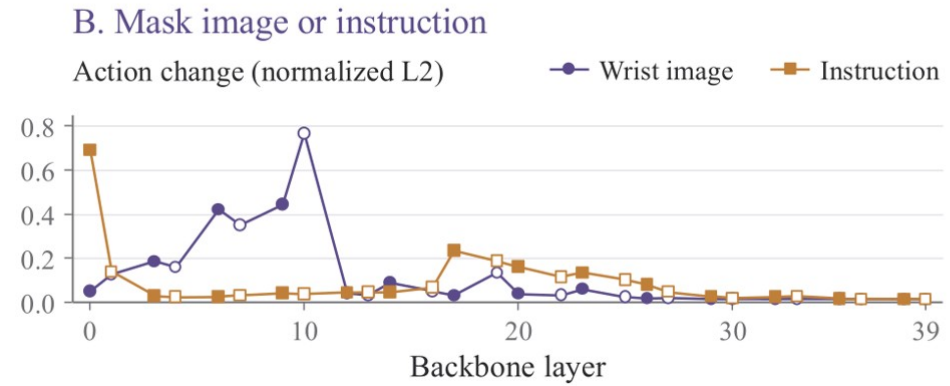
冻结时只读早期层最好；全量微调后均匀读全深度最好（95.3），end-28 始终最差。

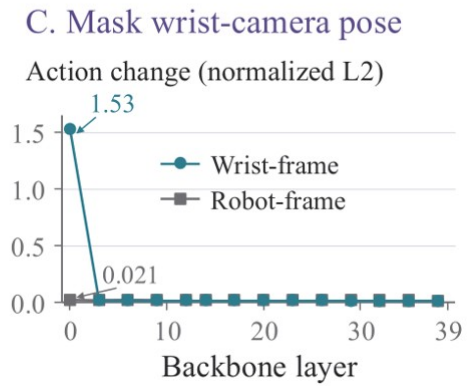
混杂因素

first-28 / first-14 同时截短了骨干（深度 28 / 14），读取位置与计算量并未完全分开。

干预实验：替换前 10 层的 KV，动作误差上升最明显







A 交换缓存 KV

第 0–10 层被替换时平移误差最高约 0.145 cm （基线≈ 0.05 cm ）； 11 层后几乎无影响。同幅度噪声的偏差 ⩽ 0.004 cm ，说明起作用的是内容。

B 遮挡图像 / 指令

指令敏感度峰值在第 0 层（≈ 0.69 ），第 17 层附近有小峰；腕部图像敏感度在第 6–10 层（峰≈ 0.77 ）。

C 遮挡腕部相机位姿

wrist-frame 模型在第 0 层响应 1.53； robot-frame 模型仅 0.021： 几何信息在最前面的层被使用，且取决于任务坐标系。

与动作相关的信息按深度分布，集中在骨干前部

保留理解能力是动作鲁棒性的前提

69.4

联合训练 · RoboTwin C2R

理解能力基本保留

10.7

仅动作微调 · RoboTwin C2R

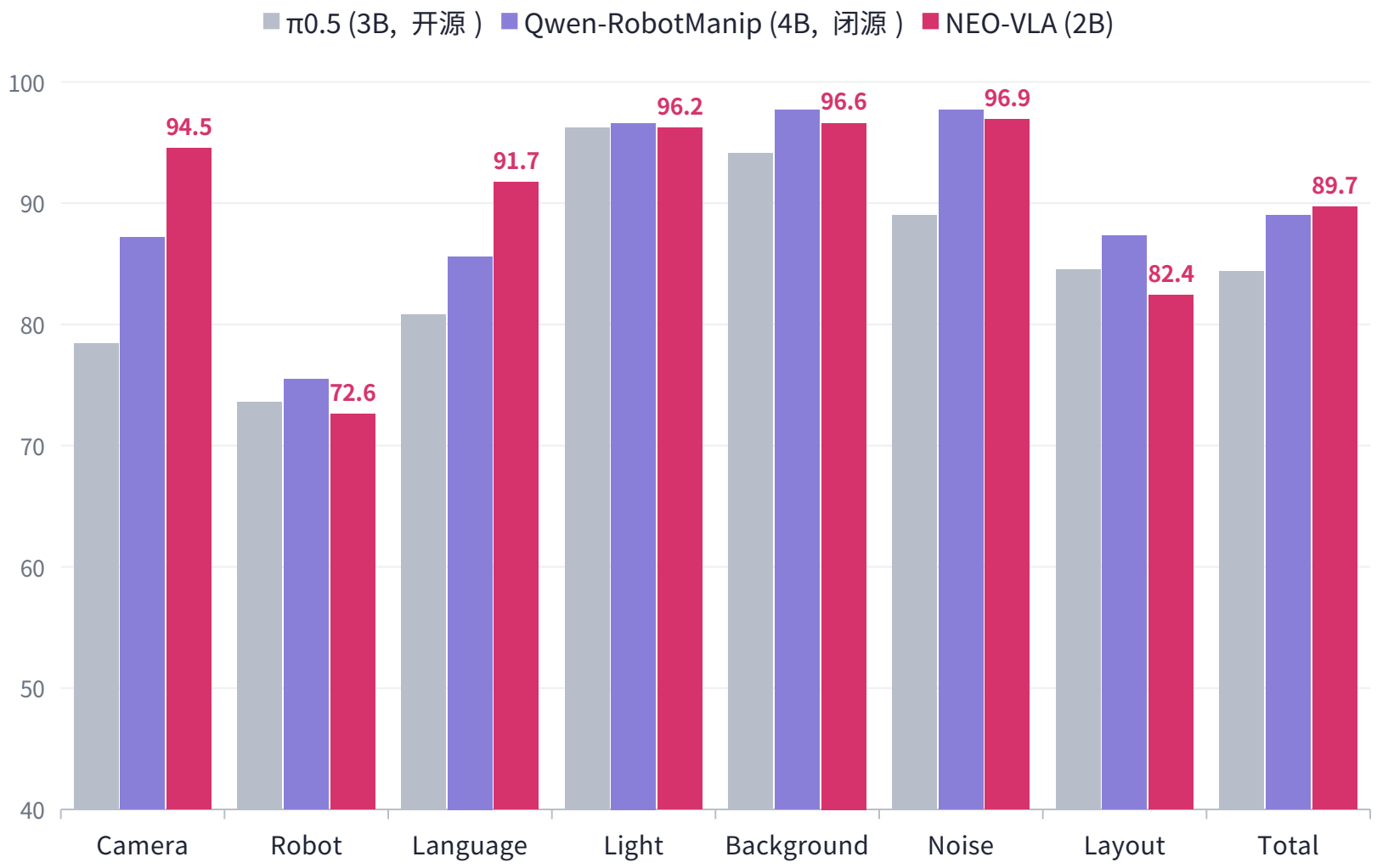
且模型无法再生成有效文本

多模态理解基准（%）：联合训练后的下降都在 1.5 分以内

基准	后训练前（Baseline）	+ 联合训练	变化
MMBench-EN	82.7	81.2	−1.5
RealWorldQA	66.8	68.4	**+1.6**
AI2D	82.9	82.3	−0.6
TextVQA	76.6	76.4	−0.2
ChartQA	80.8	80.4	−0.4

论文结论：多模态理解和鲁棒动作并不是彼此独立的能力；后训练时保留骨干的理解能力，策略泛化明显更强。

LIBERO-plus：相机与语言扰动下优势最大



89.7

总分：表中最高

Qwen-RobotManip 89.0 （闭源）

+16.1

相机扰动 94.5 vs π0.5 78.4

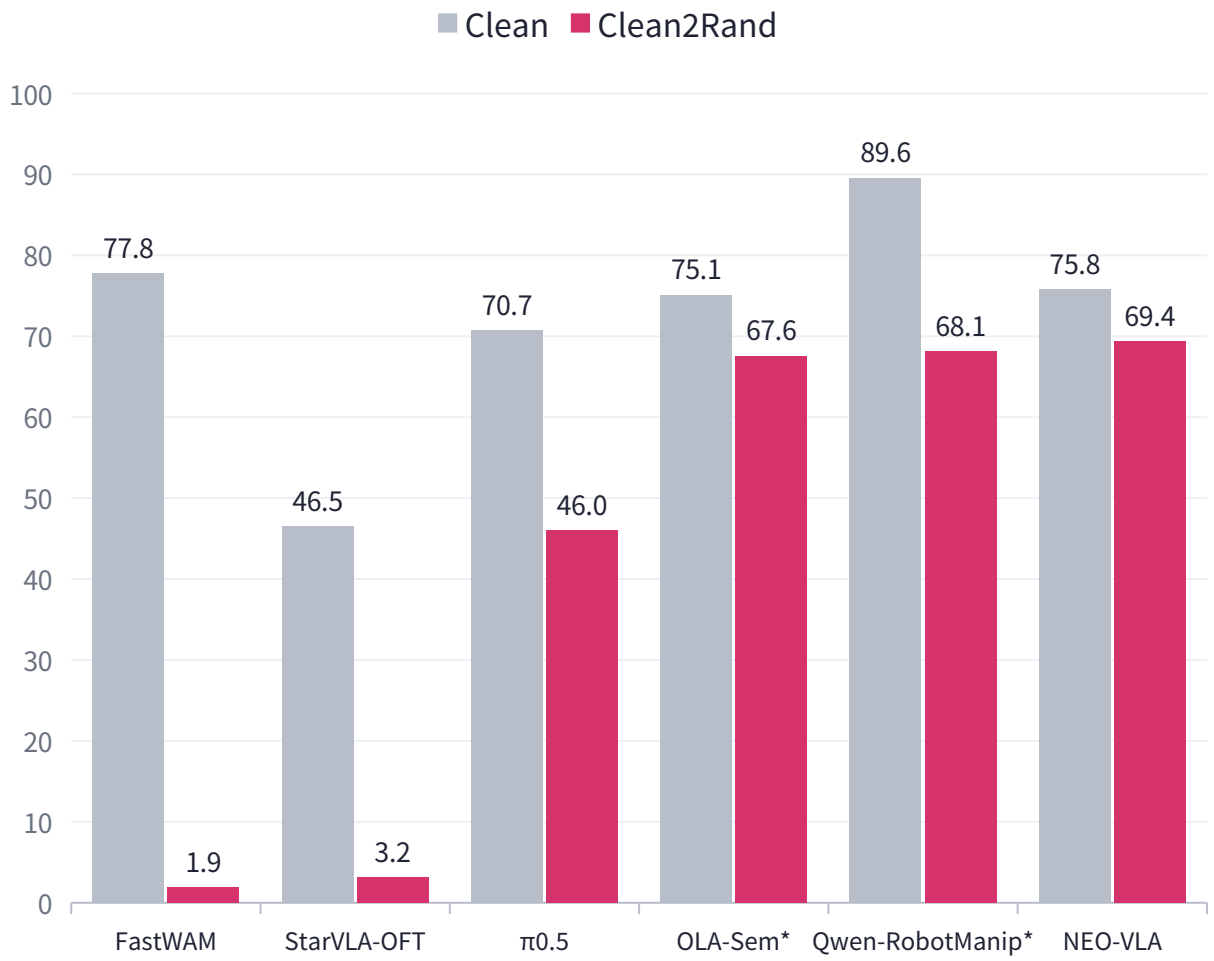
语言扰动 91.7 也是最高

短板

Robot 72.6 、 Layout 82.4 都不是最强：本体或布局本身改变时，优势减弱。

其他基准：随机化场景很稳，位置与任务泛化仍是短板

RoboTwin 2.0： Clean → Clean2Rand （* 为闭源）



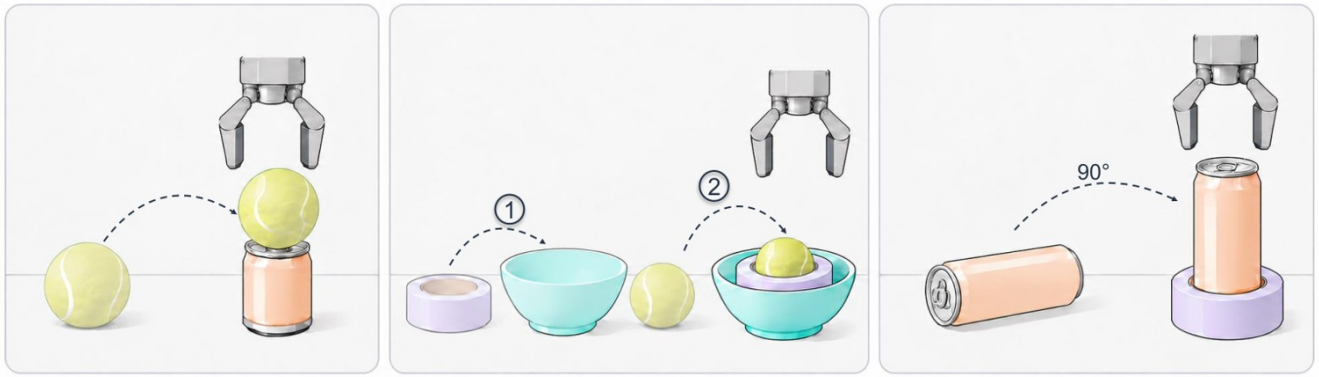
LIBERO： 已接近饱和

模型	规模	平均成功率
OpenVLA	7B	76.5
π0.5	3B	96.8
MMaDA-VLA	8B	98.0
EO-1	3B	98.4
Qwen-RobotManip （闭源）	4B	99.1
NEO-VLA	**2B**	**97.4**

LIBERO-Pro： 位置 / 任务扰动几乎失效

模型	Object	Position	Semantic	Task	Avg
OpenVLA	93.0	0.0	97.3	0.0	47.6
π0.5	96.0	20.8	95.8	0.8	53.3
X-VLA	79.0	0.8	96.8	6.8	45.8
NEO-VLA	**84.3**	**3.0**	**98.0**	**6.3**	**47.9**

真机：每个任务 50 条示教，三项任务均不低于 π0.5



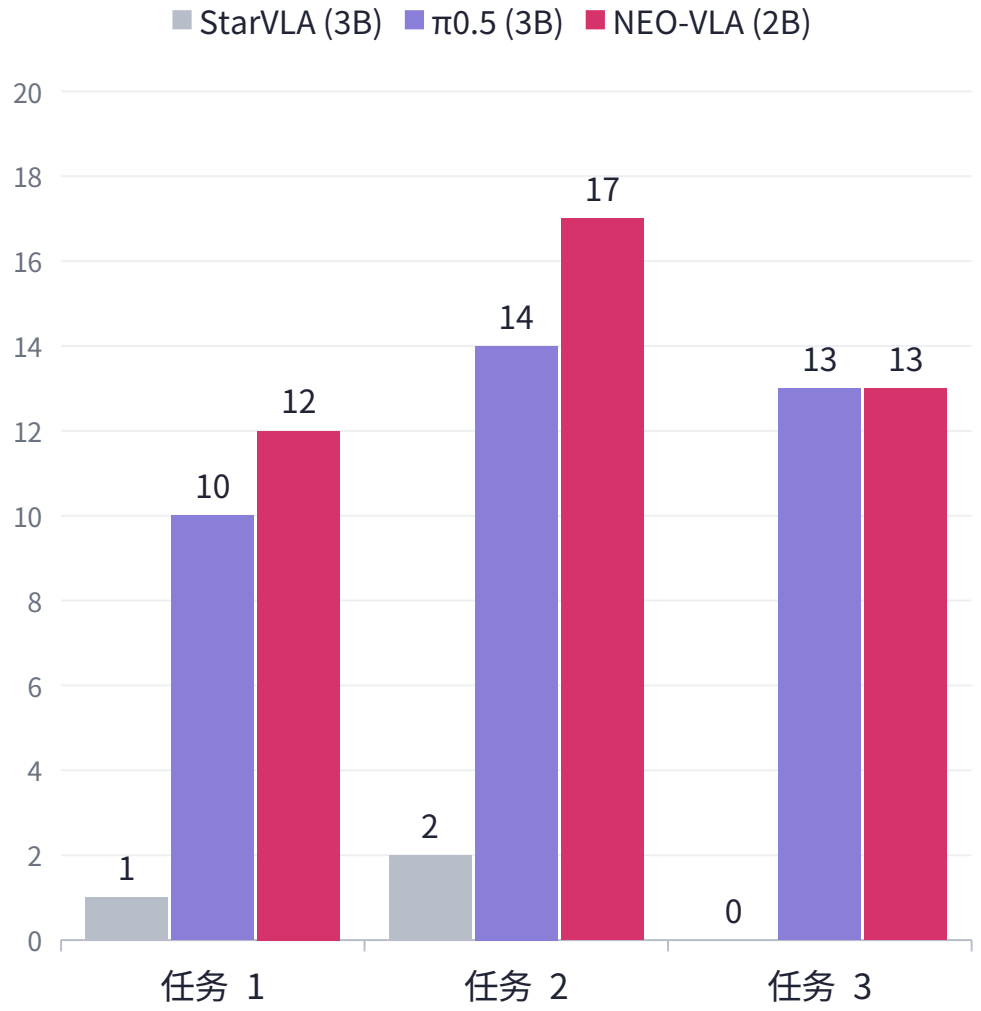
(a) Task 1: Tennis on can (b) Task 2: Tape in bowl, Tennis in tape (c) Task 3: Stand can in tape

任务 1 把网球放到罐头上（放置精度）

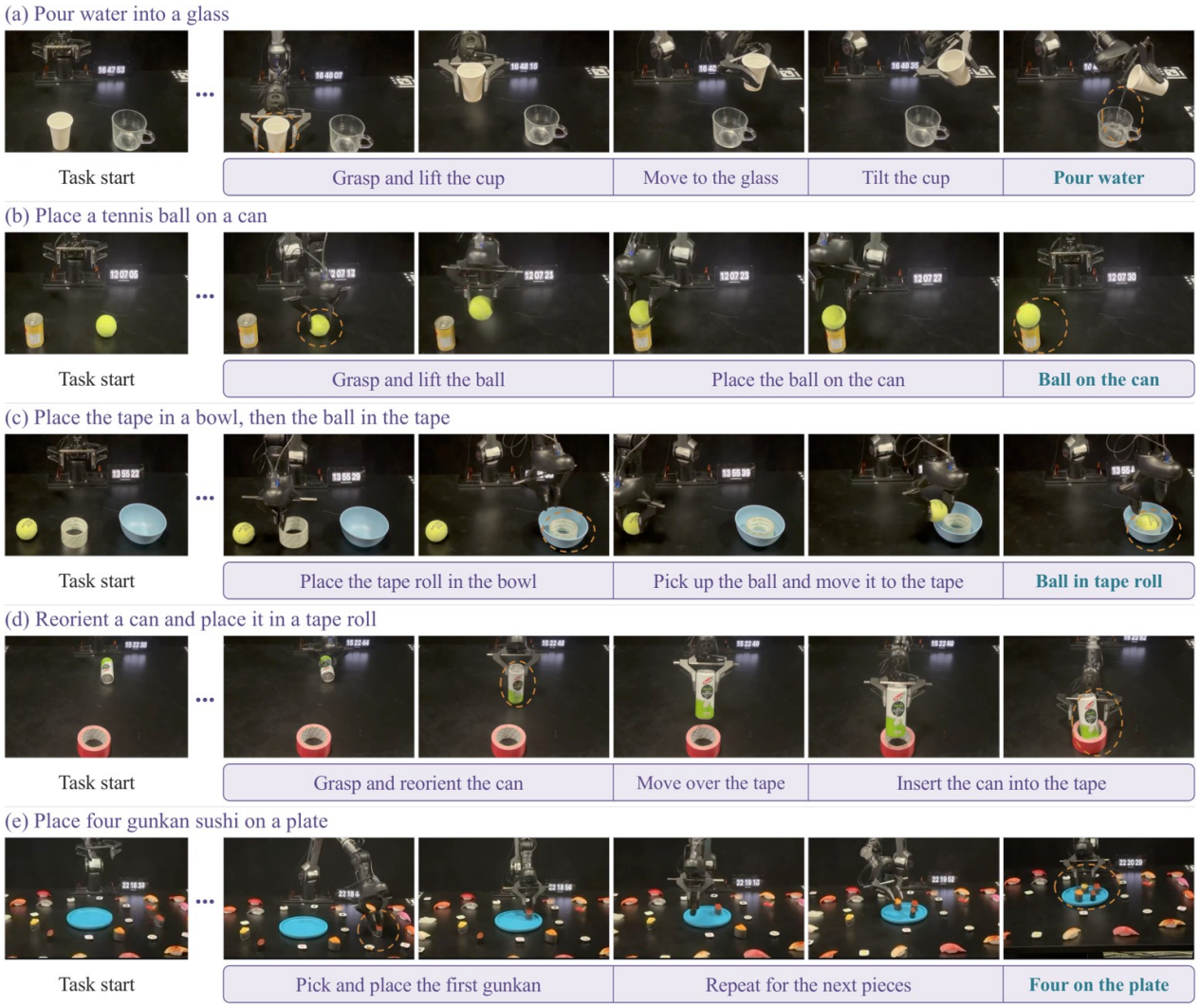
任务 2 胶带卷放进碗，再把网球放进胶带卷（多步、长时序）

任务 3 把横放的罐头旋转 90° 竖直插入胶带卷（重定向）

设置 每个任务 50 条示教微调；评测 20 次 = 10 次域内 + 10 次域外；各方法用推荐的后训练配置，推理 10 步 Euler



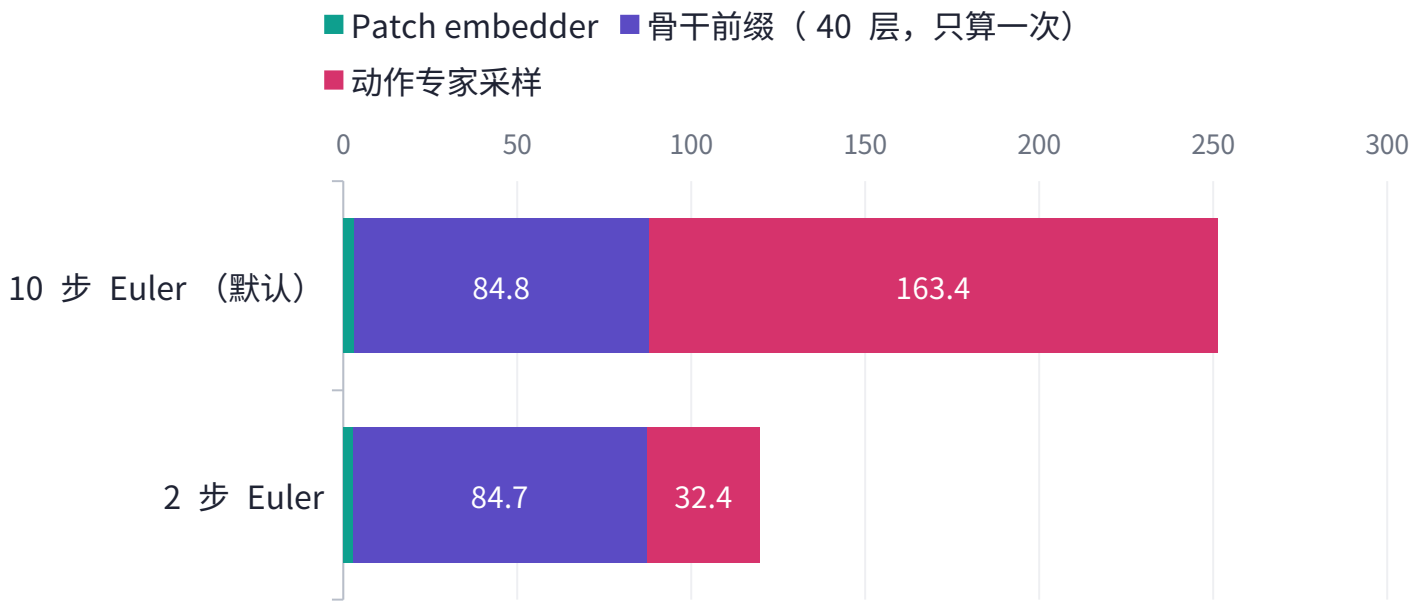
定性展示：倒水、放置、多步任务与物体重定向



- **a** 拿起纸杯，向玻璃杯倒水
- **b** 把网球放到罐头上
- **c** 胶带卷放进碗，再把网球放进卷中
- **d** 重定向横放的罐头，竖直插入胶带卷
- **e** 从混合寿司中逐个挑出 4 个军舰寿司放到盘上

注：(a)(e) 只有定性展示，论文未报告这两项任务的成功率。

推理延迟： RTX 3090 上单次查询约 0.28 秒



Patch embedder 仅 3.1 ms （ 10 步） / 2.7 ms （ 2 步），条形过窄未标数值

观察： 10 步时动作专家是主要开销；降到 2 步后，骨干前缀成为主要开销。

代价： 减少步数会改变采样出的动作，真机成功率（ Table 10 ）均在 10 步下测得。

说明： 分阶段耗时取自 Table 11 ，各项之和不等于整次前向的耗时。

端到端耗时（ ms / 次查询）

阶段	10 步	2 步
前向（编译后）	282.5	147.8
前向（ eager ）	515.4	210.7
请求 → 动作块	261.9	153.5
请求 → 可执行计划	**286.2**	**178.5**

≈0.29 s

真机：请求 → 可执行计划（ 10 步）

单张 RTX 3090 、 bf16 ，每次输出 30 步动作块，未使用实时分块（ real-time chunking ）

评价：问题提得清楚，但归因仍有混杂

亮点

1. 原生架构让“读哪一层”成为可控变量，编码器式 VLA 做不到
2. 层映射消融 + KV 替换 / 遮挡干预 + Qwen 反例，三类证据互相印证；匹配噪声对照排除了“扰动幅度”的解释
3. 鲁棒性提升集中在相机 / 噪声 / 语言扰动，与机制分析吻合
4. 权重、代码、预训练数据集全部公开；统一动作空间与数据清洗流程有工程价值

需要注意

1. 与 Qwen / Gemma 的对比混杂了预训练数据和方法差异（作者已承认）
2. first-28 冻结最好，但同时截短了骨干；干预实验只在 skip-28 模型上做
3. LIBERO-Pro 位置 3.0 、任务 6.3 ： 空间与任务层面的泛化仍弱
4. 真机每个任务仅 20 次；纯动作微调跌到 10.7 的原因未解释；单次推理约 0.28 s

三点结论

1

控制需要的视觉信息分布在各个深度

并且常在骨干的早期层达到峰值；只把编码器的最终输出交给动作头，会浪费这部分信息。

2

关键不只是去掉编码器

视觉和语言要从一开始就在同一个骨干里一起演化；只给编码器式模型开放中间层，冻结时反而更差。

3

下一个瓶颈：从感知到灵活动作

NEO-VLA 对画面变化很稳，但物体位置和任务目标一变就明显失败。

项目主页： https://www.lmms-lab.com/posts/neo_vla/ 代码： https://github.com/EvolvingLMMs-Lab/NEO
模型： huggingface.co/weepiess2383/NEO-VLA 数据： huggingface.co/datasets/weepiess2383/NEO-VLA-Action-Pretrain arXiv 2611.06918